

S4-F — Technical Documentation

Contents

S4-F — Technical Documentation	1
Table of contents	1
1. System overview	2
2. Repository layout	3
3. Module reference	3
4. Observation specification	5
5. Action pipeline and driver aids	6
6. Reward function	6
7. Episode termination	7
8. Network architectures	7
9. TORCS integration (Windows and Linux)	8
10. Training pipelines with commands	8
11. Distributed Optuna infrastructure	10
12. Checkpoints and artifacts	11
13. Configuration reference	11
14. Constraints and gotchas	12
15. Troubleshooting	12

S4-F — Technical Documentation

Wersja polska: [TECHNICAL_DOCUMENTATION.pl.md](#)

Neural racing agent for TORCS/SCR (IBM AI Racing League), Corkscrew circuit. This is the complete technical reference for the repository: system architecture, module map, observation/action/reward specification, TORCS integration on Windows and Linux, the full training pipeline with commands, checkpoint formats, and troubleshooting.

Companion documents:

Document	Content
case_study.pdf	Full engineering case study (methodology, experiments, results)
EXPERIMENT_LOG.md	Chronological engineering log (raw, unedited)
physics_desync_and_variance_collapse_bug_2026-06-30.md	Post-mortem of the residual-RL failure cascade
DISTILL_COMMANDS.md	Copy-paste runbook for the distillation pipeline
README_WSL.md	WSL headless TORCS build guide (original notes)

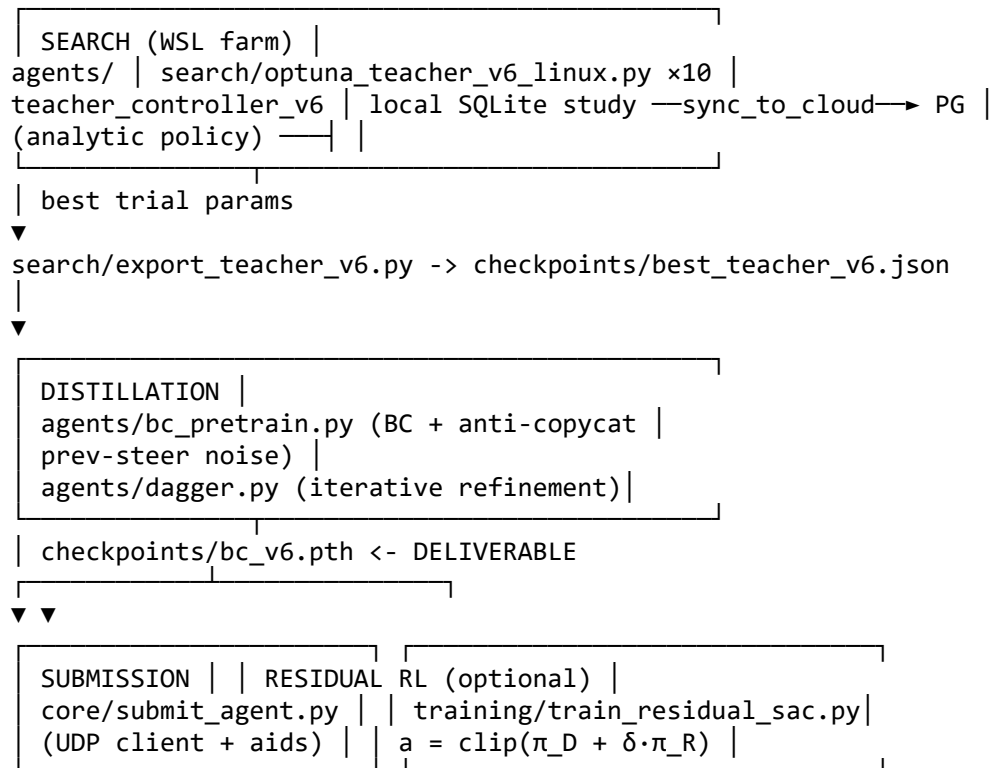
Table of contents

- 1. System overview

2. Repository layout
 3. Module reference
 4. Observation specification
 5. Action pipeline and driver aids
 6. Reward function
 7. Episode termination
 8. Network architectures
 9. TORCS integration (Windows and Linux)
 10. Training pipelines with commands
 11. Distributed Optuna infrastructure
 12. Checkpoints and artifacts
 13. Configuration reference
 14. Constraints and gotchas
 15. Troubleshooting
-

1. System overview

The system produces a neural-network driver through a **teacher-student pipeline**: an analytic, rules-based racing controller (“teacher”) with 54 tunable parameters is optimised by a distributed Optuna search, then compressed (“distilled”) into a small multilayer perceptron via behavioural cloning and DAgger. Model-free reinforcement learning (Soft Actor-Critic) serves two auxiliary roles: from-scratch training baselines, and *residual* fine-tuning of the frozen distilled policy.



Runtime interface: the SCR server inside TORCS exchanges UDP packets with `core/snakeoil3_gym.py` at 50 Hz (20 ms per tick, port 3001 + driver index). Every consumer builds observations with `core/observation_utils.py` and post-processes actions with `core/driving_aids.py`, so training-time and submission behaviour are **identical by construction**.

Headline result: `checkpoints/bc_v6.pth` (108,674 parameters, 429 KB) scored a **92.04 s** standing-start lap on

Corkscrew under submission conditions (fuel + damage enabled), top speed 250 km/h. A residual reinforcement-learning policy on the same frozen base (checkpoints/residual_sac_latest.zip) reaches a comparable ~92 s lap, so the result holds under RL and not imitation alone — both agents ship.

2. Repository layout

```
S4-F/
├── README.md # project overview + quick start
├── S4-F_requirements.txt # pinned Python dependencies
├── config.py # <- single source of truth for ALL configuration
├── practice.xml # race-config template (MUST stay at root, §14)
├── autostart_win.py # Windows GUI race auto-start (MUST stay at root, §14)
├── .gitignore
├──
├── core/ # runtime: submission chain + training environment
├── agents/ # analytic teachers (v1-v6) + BC / DAgger / BC-anchored SAC
├── search/ # distributed Optuna infrastructure (all versions)
├── training/ # SAC / residual-SAC training drivers
├── tools/ # diagnostics, evaluation, telemetry analysis
├── phase2/ # stage-2 (multi-car / mass-start) pipeline
├──
├── checkpoints/ # model artifacts (bc_v6.pth = deliverable)
├── docs/ # this file, case study, experiment log, guides
├── scripts/ # legacy Windows .bat launchers (provenance)
```

Modules are Python **packages**: run entry points from the repository root with `python -m <package>.<module>`, e.g. `python -m core.submit_agent`. (`config.py` sits at the root, so `import config` resolves for every entry point started from the root; see §14.)

3. Module reference

3.1 core/ — runtime (submission chain + environment)

Module	Role
<code>core/submit_agent.py</code>	Competition entry point: loads BCNetwork weights, drives over UDP with driver aids
<code>core/snakeoil3_gym.py</code>	SCR UDP client: connect, parse telemetry, format commands, TORCS relaunch
<code>core/observation_utils.py</code>	raw telemetry dict -> normalised 32/68-dim float32 vector
<code>core/driving_aids.py</code>	NN action -> TORCS command: RPM gear scheduler, TCS, launch centering
<code>core/torcs_env_sac.py</code>	Gymnasium env: TORCS launch (Windows GUI / Linux headless), UDP step loop, termination, reward hookup
<code>core/reward_functions.py</code>	stage-1 / stage-2 reward computation (§6)
<code>core/custom_policy.py</code>	LayerNormSACPolicy — SAC actor/critic with LayerNorm after every hidden layer
<code>core/telemetry_recorder.py</code>	per-tick car CSV + per-100-step training-stats CSV
<code>core/callbacks.py</code>	SB3 callbacks: checkpointing, entropy decay, actor freeze

3.2 agents/ — policies

Module	Model	Status
agents/teacher_controller_v6.py	decoupled braking envelope + out-in-out racing line + near-redline rev-limit shifting	current teacher
agents/teacher_controller_v5.py	6 minus racing line	superseded
agents/teacher_controller_v4.py	$\square \sqrt{\text{reach}}$ (coupled straight/corner)	legacy
agents/teacher_controller_v3.py	12+12 static waypoints; also hosts evaluate_teacher used by search	legacy but load-bearing
agents/teacher_controller_v2.py	lookahead PID variants	legacy
teacher_controller.py		
agents/tune_teacher.py	v1/v2 Optuna tooling + params JSON I/O	legacy
agents/bc_pretrain.py	teacher rollout collection + BCNetwork + weighted-MSE BC training with prev-steer noise	current
agents/dagger.py	DAGger loop (teacher-relabelled aggregates, seeded with demos)	current
agents/bc_anchored_sac.py	BCAnchoredSAC — SAC subclass with TD3+BC-style mean-only anchor	current

Teacher interface contract: `TeacherController(params).act(raw_obs) -> [steer, accel_brake]`, `get_gear() -> int`; per version: `sample_params(trial), params_from_optuna(dict)`.

3.3 search/ — distributed Optuna farm

Module	Role
search/optuna_teacher_linux.py	shared plumbing: <code>install_practice_xml(port)</code> , <code>headless launch_torcs (torcs -r)</code> , <code>scoped kill_torcs</code>
search/optuna_teacher_v6_linux.py	per-worker trial loop for v6 (<code>ask -> race -> tell</code>), stuck detection
search/optuna_teacher_v{2,4,5}_linux.py	same for older controllers (legacy)
search/optuna_teacher_v3.py	<code>make_storage()</code> (WAL SQLite / pooled PostgreSQL) — used by every driver ; v3 trial tooling
search/optuna_teacher_single.py	Windows single-machine sweep (legacy)
search/new_study_v6.py (<code>_v2/_v4/_v5, new_study.py</code>)	study creation + seed-trial enqueue
search/export_teacher_v6.py	best trial -> <code>checkpoints/best_teacher_v6.json</code>
search/sync_to_cloud.py	one-way local-SQLite -> PostgreSQL mirror
search/monitor.py	live terminal dashboard for the farm
search/pg_setup.py, pg_new_study.py, inspect_pg.py, save_v2_best.py	PostgreSQL administration / study utilities

3.4 training/ — RL drivers

Module	Role
training/train_residual_sac.py	residual SAC on a frozen base: seeding with residual = 0, floor checks, --eval mode. --resume none required when the base changes
training/residual_env.py	ResidualTorcsEnv: wraps TorcsSACEnv, holds the frozen base, applies $\text{clip}(\pi_D + \delta \cdot \pi_R, -1, 1)$
training/train_sac.py	from-scratch / fine-tune SAC, two-stage curriculum, transfer_stage1_to_stage2 (32->68 weight surgery)
training/train_sac_v2.py	from-scratch SAC with speed-cap curriculum (legacy)
training/optuna_sac.py, optuna_residual.py	HPO over SAC / residual hyperparameters (legacy)
training/multi_instance_torcs.py	Windows multi-instance TORCS manager, ports 3001–3006 (legacy era)

3.5 tools/ — diagnostics

Module	Role
tools/eval_policy.py	closed-loop deterministic/stochastic evaluation of a .pth policy
tools/telemetry_lap.py	speed-vs-distance profile of a study's best parameters (the tool that found the 174 km/h cap)
tools/diag_lap.py	single-lap sanity check / --watch graphical replay
tools/inspect_best.py	print best-checkpoint metadata
tools/gen_scr_server.py	generate SCR server config XML
tools/granite_analysis.py	IBM Granite LLM per-corner telemetry commentary (optional; local Ollama or watsonx)

3.6 phase2/ — stage-2 (multi-car) pipeline

Self-contained mass-start variant: race_config.py (opponent grid XML generation), residual_env_stage2.py (68-dim observations), train_mass_start.py, submit_agent_stage2.py. Not part of the scored time-trial deliverable.

4. Observation specification

Stage 1 (time trial): 32 dimensions. Stage 2 appends 36 opponent rangefinders (/200 m) -> 68 dimensions; indices 0–31 are a strict prefix, so stage-1 weights transfer loss-free (training/train_sac.py::transfer_stage1_to_stage2).

Index	Signal	Raw range	Normalisation
0–18	track-edge rangefinders $d_1 \dots d_{19}$ (beam angles $-45^\circ \dots +45^\circ$)	0–200 m	/200, clip [0, 1]
19, 20	speedX, speedY	± 300 km/h	/300
21	heading error ψ vs. track tangent	$[-\pi, \pi]$ rad	/ π
22	trackPos p (0 = centre, ± 1 = edges)	$\approx [-1.5, 1.5]$	clip $[-1, 1]$
23	engine speed	0–18,700 rpm	/10,000, clip [0, 1]
24	gear	1–6 (SCR range)	/7 (fixed training constant)
25–28	wheel angular velocities (FL, FR, RL, RR)	0–100 rad/s	/100

Index	Signal	Raw range	Normalisation
29	lap progress distFromStart	0–3,602 m	/3,602
30	current lap time	0–120 s	/120
31	previous steering command	[–1, 1]	identity

Index 31 caveat. `prev_steer` keeps the process Markov under the steering dynamics of the aids layer, but it enables the *copycat shortcut* in behavioural cloning (`steer ≈ prev_steer`, Codevilla et al. 2019). `agents/bc_pretrain.py::train_bc` therefore adds Gaussian noise ($\sigma = 0.15$, clipped) to this feature in every training minibatch. Deployment-time observations are unchanged.

5. Action pipeline and driver aids

```

NN output [steer, accel_brake] ∈ [–1, 1]2
-> core/driving_aids.apply_aids():
1. steering-rate limiter (disabled by default)
2. launch centering (v < 40 km/h AND lap time < 8 s:
  steer = clip(0.30·trackPos, ±0.4))
3. accel/brake split (accel_brake ≥ 0 -> throttle,
  < 0 -> brake = |accel_brake|)
+ TCS: if (rear – front) wheel-spin > 5 rad/s and v ≥ 30 km/h,
  throttle *= max(0.1, 5/slip)
4. automatic gearbox (pure RPM: up > 17,800 rpm & g < 6;
  down < 9,000 rpm & g > 1)
-> TORCS command {steer, accel, brake, gear}

```

Gear range note. The SCR protocol accepts gear commands **–1...6 only**; `core/snakeoil3_gym.py` resets any other value to **neutral**. The `car1-ow1.xml` car definition lists a seventh forward ratio, but it is unreachable over SCR. Telemetry across 2.17M recorded ticks confirms the car reaches its ~255 km/h top speed in **fifth** gear at ~17,200 rpm (sixth is engaged on <0.1 % of ticks); the decisive fix was the shift *point*, not the gear count.

The **same function** runs in training (`core/torcs_env_sac.step`), teacher data collection, evaluation and submission — shift points and TCS can never drift between data collection and deployment. This mattered: an earlier 8,200-rpm short-shift schedule in this very layer capped every agent at 174 km/h (see `docs/EXPERIMENT_LOG.md` §8 and the case study).

6. Reward function

Stage 1 (`core/reward_functions.py::reward_stage1`), per 20 ms tick:

```

r = 1.0 · Δ distRaced # progress [m]; clamped to [0, 50]
  - 0.1 # per-step time cost
  - 0.05 · trackPos2 # centring (quadratic)
  - 0.05 · |angle| # heading alignment
  - 0.1 · |Δ steer| # steering smoothness
  + 0.2 · (speedX/300) if min(track[8:11]) > 150 # straight-line bonus
  + 1.0 · (speedX/300) · |cos ψ| · (1 – |p|) · sharp # corner-speed reward
  - 0.025 · min(Δ damage, 200) # damage
  + events

```

where `sharp = max(0, 1 – min(track[7:12])/150)` and events are: off-track –1 per step, backwards –10, lap completion +500 + `max(0, 90 – lap_time)`. Stage 2 adds an opponent-proximity penalty (linear inside 10 m), +5 per overtake, –5 per contact.

Design note: an earlier $\text{speed} \cdot \cos(\psi)$ reward was abandoned in S2 — its integral is also just distance, so crawling and racing earn similar returns over a fixed horizon. The progress + time-cost + lap-bonus form makes lap time explicit in the return.

7. Episode termination

From `core/torcs_env_sac.py` (thresholds in `config.TorcsConfig`):

Condition	Threshold
off-track	$ \text{trackPos} > 1.10$ for > 30 consecutive ticks (0.6 s grace — kerbs allowed)
reversed	$\cos(\text{angle}) < 0$
damage	cumulative $> 5,000$
stuck	forward progress < 2 m over 250 ticks
timeout	9,000 ticks (180 s)
connection	UDP silence / TORCS process death

Do not re-add a $\min(\text{track}) < 0$ clause to the off-track predicate: angled rangefinders legitimately return -1 (no surface within 200 m) in long, tight corners. This exact clause silently truncated training episodes at the $\sim 2,400$ m hairpin for every agent generation until it was removed (documented defect; see the case study §5.4.3).

8. Network architectures

8.1 BCNetwork (the deliverable)

Defined in `agents/bc_pretrain.py`; mirrors the SAC actor trunk exactly:

```
input 32 -> Linear 256 -> LayerNorm -> ReLU
-> Linear 256 -> LayerNorm -> ReLU
-> Linear 128 -> LayerNorm -> ReLU
-> Linear 2 -> tanh -> [steer, accel_brake]
```

108,674 parameters, 429 KB serialised (`torch.save(state_dict)`). Loading: `torch.load(path, map_location="cpu", weights_only=True)`.

8.2 SAC actor-critic (`core/custom_policy.py`)

`LayerNormSACPolicy` — Stable-Baselines3 `SACPolicy` with `LayerNorm` inserted after every hidden `Linear` in both the actor latent MLP and each of the two Q-networks (double-Q, min-target). Hidden sizes [256, 256, 128] for both actor and critics (`config.SACConfig.pi_layers/qf_layers`). `LayerNorm` was adopted in S2 after a critic divergence and retained everywhere since.

8.3 Principal hyperparameters

Parameter	Stage-1 SAC	Residual SAC
learning rate	$3e-4$	$1e-4$
replay buffer	1,000,000	500,000
batch size	256	256
γ / τ	0.99 / 0.005	0.99 / 0.005
entropy coef	auto (target -2)	0.02 fixed
gSDE	on, resample every step	on, resample every step
train schedule	per step	per episode (<code>gradient_steps = -1</code>)
BC anchor β_0 / decay	100 / 300k steps	100 / 500k steps

Parameter	Stage-1 SAC	Residual SAC
warmup (learning_starts)	10,000	seeded with 20k base-policy steps
residual bounds δ	—	(0.15 steer, 0.50 accel/brake)

Two of these values are safety-critical, not preferences: `sde_sample_freq = 1` (an 8-tick noise hold = 160 ms = 6 m of travel at speed -> crash) and per-episode training (backprop inside the 20 ms control window desynchronises the physics; see the post-mortem doc).

9. TORCS integration (Windows and Linux)

9.1 Protocol

SCR (Simulated Car Racing) over UDP, 50 Hz. Telemetry in: 23 fields (angle, speeds, 19 track beams, 36 opponent beams, rpm, gear, fuel, damage, distFromStart/distRaced, lap times, wheel spins, trackPos, z, racePos). Commands out: steer, accel, brake, gear, meta. A response must arrive within the 20 ms tick or TORCS reuses the previous command.

9.2 Windows (GUI)

1. Install TORCS 1.3.7 + SCR patch (see README \$Installation). Expected paths (edit `config.py` if different):
`TORCS_EXE = D:\torcs\torcs\wtorcs.exe`, `TORCS_CONFIG_DIR = D:\torcs\torcs`.
2. Start a race manually (Race -> Practice -> New Race with the `scr_server` driver) **or** let the code do it: `core/torcs_env_sac` launches `wtorcs.exe` and drives the menu via `autostart_win.py` (pyautogui keystrokes; the TORCS window must be focusable).
3. The client connects on UDP port 3001 (driver index 0).

9.3 Linux / WSL (headless)

Build TORCS 1.3.7 from the `fmirus/torcs-1.3.7` source (identical physics to the competition container) and install the SCR server driver — full recipe in [README_WSL.md](#). Key facts:

- `torcs -r <race.xml>` with `display mode = results only` runs without 3D rendering: CPU-bound, can run faster than real time.
- `search/optuna_teacher_linux.py::install_practice_xml(port)` copies the repo-root `practice.xml` into `~/.torcs/config/raceman/` and patches the SCR driver index (= `port - 3001`) per worker.
- Each parallel worker needs its **own HOME** (`HOME=~/.torcs_w<k>`) so `~/.torcs` state does not collide.
- First launch on a cold HOME segfaults; `TorcsSACEnv` performs a one-time warm-up (`timeout 10 torcs`) and retries connects (up to 4×).
- The compiled SCR server exposes **10 robot slots** (ports 3001–3010) — a hard per-machine parallelism cap.
- The WSL farm launches with `-nofuel -nodamage -nolaptime`; the submission path runs **with** fuel and damage. Expect ≈ 0.7 s lap-time difference; report submission-path figures as primary.

9.4 Race configuration

`practice.xml` (repo root): Corkscrew, road category, practice session, 1 driver (`scr_server` idx 0), standing start. It is *copied*, never read in place — but the copy source is resolved relative to the repo root, so the file must not move.

10. Training pipelines with commands

All commands run **from the repository root**. Windows: use `python`; WSL: `python3` inside the venv. Full annotated runbook: [DISTILL_COMMANDS.md](#).

10.0 Submission (reproduce the scored lap)

core/submit_agent.py runs one of two selectable policies, both through the same driving_aids post-processing and both reaching a ~92 s standing-start lap — the deterministic BC/Dagger deliverable (92.04 s) and a residual SAC policy that adds reinforcement-learning corrections on top of it:

```
# BC – the scored 92.04 s deliverable (default):
```

```
python -m core.submit_agent --weights checkpoints/bc_v6.pth --episodes 1 --port 3001
```

```
# Residual – SAC corrections on a frozen base: clip(base +  $\delta$ ·residual).
```

```
# The shipped residual was trained on bc_v6.pth (the default --base):
```

```
python -m core.submit_agent --residual checkpoints/residual_sac_latest.zip --episodes 1 --port 3001
```

The residual path loads the SB3 SAC .zip with custom_objects={"policy_class": LayerNormSACPolicy} (this neutralises the module-path change from repackaging), reads δ from Config.residual (delta_steer=0.15, delta_accel=0.50), and combines it with the frozen --base network exactly as training/residual_env.py::compute_final_action does. **--base must match the network the residual was trained on** (the shipped residual_sac_latest.zip was trained on bc_v6.pth, the default). No VecNormalize is needed at inference (the residual eval path uses a passthrough VecNormalize(norm_obs=False, norm_reward=False)). The residual checkpoint's 138 MB replay buffer is not shipped — only residual_sac_latest.zip.

10.1 Teacher search (WSL farm)

```
# one-time: create the study + enqueue seeds
```

```
python -m search.new_study_v6 --study-name teacher_v6_ow1 \
--storage sqlite:///home/user/optuna_ow1.db
```

```
# per worker k = 0..9
```

```
HOME=~/.torcs_w$k python3 -m search.optuna_teacher_v6_linux \
--study-name teacher_v6_ow1 \
--storage sqlite:///home/user/optuna_ow1.db \
--port $((3001+k)) --n-trials 1000000
```

```
# live dashboard / cloud mirror (optional)
```

```
python -m search.monitor
python -m search.sync_to_cloud
```

```
# export the best trial
```

```
python -m search.export_teacher_v6 --study-name teacher_v6_ow1 \
--storage sqlite:///home/user/optuna_ow1.db \
--output checkpoints/best_teacher_v6.json
```

10.2 Distillation (BC -> DAgger)

```
# smoke test (3k steps – expect a ~88 s Lap within a minute)
```

```
python -m agents.bc_pretrain --controller v6 \
--teacher-params checkpoints/best_teacher_v6.json \
--n-steps 3000 --output checkpoints/_bc_smoke.pth
```

```
# full BC (500k transitions)
```

```
python -m agents.bc_pretrain --controller v6 \
--teacher-params checkpoints/best_teacher_v6.json \
--n-steps 500000 --output checkpoints/bc_v6.pth \
--prev-steer-noise 0.15
```

```
# Dagger refinement (5 × 100k, teacher-seeded)
python -m agents.dagger --controller v6 \
  --teacher-params checkpoints/best_teacher_v6.json \
  --bc-weights checkpoints/bc_v6.pth \
  --iterations 5 --steps-per-iter 100000 --epochs 30 \
  --output checkpoints/dagger_v6.pth
```

```
# validate
python -m tools.eval_policy --bc-pretrain checkpoints/bc_v6.pth --episodes 3
```

Do **not** run BC/Dagger collection on a machine with live Optuna workers: collection uses port 3001 and a scoped TORCS kill on reset. Pause the workers or use a different machine.

10.3 Residual SAC (optional refinement)

```
# fresh run against a given frozen base (--resume none is mandatory
# whenever the base network changed)
python -m training.train_residual_sac \
  --dagger-weights checkpoints/bc_v6.pth \
  --resume none --skip-floor-check --ent-coef-final 0.005

# evaluate a residual checkpoint against its base
python -m training.train_residual_sac \
  --eval checkpoints/residual_sac_<N>_steps.zip \
  --dagger-weights checkpoints/bc_v6.pth # Look for "IMPROVED on DAGger"
```

10.4 From-scratch SAC (baseline lineage)

```
python -m training.train_sac --stage 1 --timesteps 5000000 \
  --seed-demos 20000 --bc-pretrain checkpoints/bc_v6.pth
```

10.5 Stage 2 (multi-car, not scored)

```
python -m phase2.train_mass_start # see phase2/README.md
python -m phase2.submit_agent_stage2 --weights <stage2 ckpt>
```

11. Distributed Optuna infrastructure

- **Storage:** `search/optuna_teacher_v3.py::make_storage()` builds either a WAL-mode SQLite storage (`busy_timeout`, `skip_compatibility_check`) or a pooled PostgreSQL storage pinned to `pool_size=1`, `max_overflow=0` per process (cloud pool exhaustion fix).
- **Topology used:** each machine ran its workers against **local SQLite**; `search/sync_to_cloud.py` mirrored completed trials one-way into a cloud PostgreSQL study (skipping individual out-of-distribution seed trials rather than aborting the batch).
- **Scale achieved:** ~15,000 trials across studies (v3: 2,210 → 118.8 s plateau; v5b/v6 converged to 91.2 s / ≈88.8 s with 10 workers/machine).
- **Trial anatomy:** ask params → `install_practice_xml(port)` → headless torcs -r → teacher drives 1–3 laps over UDP → best lap reported → `tell(study)`. Wedged-but-on-track cars are cut by a no-progress bailout (~4 s without forward distance).
- **Failure modes fixed** (details in `EXPERIMENT_LOG.md` §2, §7, §10): global `pkill torcs` in the client self-heal cascading across workers (now scoped per worker); workers exiting silently at a study's `--n-trials`; per-trial 300 s timeouts from stuck cars.

12. Checkpoints and artifacts

File	Format	Content
checkpoints/bc_v6.pth	PyTorch state dict	DELIVERABLE — distilled policy; 92.04 s scored lap
checkpoints/dagger_policy_v2.pth	PyTorch state dict	fallback policy (106.96 s, 6/6 clean episodes)
checkpoints/best_teacher_v6.json	JSON (54 params)	frozen tuned teacher — full provenance/reproduction
checkpoints/best_teacher_v2_pg.json	JSON (8s.json)	historical teacher snapshot
*_steps.zip + *_vecnorm*.pkl (gitignored)	SB3	SAC training checkpoints (policy+optimizer / VecNormalize stats)

Loading the deliverable:

```
from agents.bc_pretrain import BCNetwork
import torch

net = BCNetwork(obs_dim=32, action_dim=2, hidden_sizes=[256, 256, 128])
net.load_state_dict(torch.load("checkpoints/bc_v6.pth",
    map_location="cpu", weights_only=True))
net.eval()
action = net(obs_tensor) # [steer, accel_brake] ∈ [-1, 1]2
```

(A 31-dim legacy checkpoint is auto-padded with a zero prev_steer column by training/residual_env.py::_load_dagger)

13. Configuration reference

Everything lives in config.py as dataclasses under the Config singleton:

Field	Contents	Critical values
Config.torcs	paths, ports, track, termination thresholds	track_length_m=3602, off-track_trackpos_threshold=1.10, max_steps_per_episode=9000, base_port=3001
Config.observation (alias obs)	normalisation constants, dims	stage1_dim=32, stage2_dim=68, gear_max=7.0
Config.aids (alias action)	gear/TCS/launch parameters	rpm_upshift=17800, rpm_downshift=9000, max_gear=6 (SCR clamp), tcs_slip_threshold=5.0
Config.reward	all reward weights of \$6	lap_bonus=500, lap_target_time=90
Config.sac	SAC nets + hyperparameters	pi_layers=qf_layers=[256, 256, 128]
Config.residual	residual RL	delta_steel=0.15, delta_accel=0.50, train_freq=(1, "episode"), gradient_steps=-1
Config.teacher_v3	v3 waypoint teacher defaults	12 speed + 12 line waypoints
Config.multi	Windows multi-instance	n_instances=6, base_port=3001
Config.training / Config.curriculum	schedules, seeding, stage-2 ramp	seed_steps=50000

TempConfig({...}) is a context manager that temporarily overrides dotted config paths for the scope of an Optuna trial.

14. Constraints and gotchas

1. **Run entry points from the repo root** (python -m package.module). config.py is a root-level module: any process whose sys.path[0] is the repo root resolves import config; running a file from inside a package directory will not.
2. **practice.xml must stay at the repo root.** search/optuna_teacher_linux.py resolves it one directory above its own location and copies it into ~/.torcs on every headless launch.
3. **autostart_win.py must stay at the repo root.** It is invoked by absolute repo-root path from core/torcs_env_sac.py / core/snakeoil3_gym.py, and by training/multi_instance_torcs.py.
4. **Off-track termination uses |trackPos| > 1.10 only** (§7). Do not “harden” it with rangefinder sign checks.
5. **--resume auto in residual training loads the newest checkpoint on disk.** After changing the base network, pass --resume none or clear old residual_sac_* checkpoints.
6. **Per-worker HOME on WSL** (§9.3) and the **10-robot SCR cap** are hard operational limits.
7. **Windows vs WSL flag parity:** the farm ran -nofuel -nodamage; submission runs with both enabled (≈ 0.7 s slower). Quote submission-path numbers.
8. **Secrets:** the PostgreSQL DSN lives in an untracked .pg_url file (gitignored). Never commit it; search/sync_to_cloud.py expects it at the repo root when cloud mirroring is used.
9. **Do not backpropagate inside the control loop.** If you change training schedules, keep gradient updates off the 20 ms real-time path (train_freq=(1, "episode")); see the post-mortem document for the failure signature.

15. Troubleshooting

Symptom	Likely cause -> fix
ModuleNotFoundError: config	Entry point not started from the repo root. cd to the root and use python -m package.module.
Client hangs on Waiting for server on 3001	TORCS not running / no SCR driver in the race / wrong port. Start Practice with scr_server idx 0, or check --port.
torcs -r segfaults on first WSL launch	Cold ~/.torcs. Run the one-time warm-up (timeout 10 torcs) or just retry — TorcsSACEnv does both automatically.
Optuna trials all report 999 / no laps	Teacher params out of range or TORCS instance dead. Run one lap manually: python -m tools.diag_lap --port 3001.
Workers exit silently after N trials	Study reached its --n-trials. Relaunch with --n-trials 1000000.
max clients reached ... pool_size (PostgreSQL)	Too many pooled connections. Use make_storage() (pins pool to 1/process) or move workers to local SQLite + sync_to_cloud.
Every BC episode ends near 2,463 m	The min(track) < 0 clause was re-introduced into termination. Remove it (§7, §14.4).
BC validation loss excellent, car crashes in closed loop	Copypcat shortcut on prev_steer. Train with --prev-steer-noise 0.15 (default).
Car brakes late and crashes only in training mode	Gradient updates on the control path. Restore train_freq=(1, "episode"), gradient_steps=-1.
Residual policy instantly worse than its base	Resumed a checkpoint trained against a different base. --resume none.

Symptom	Likely cause -> fix
Top speed stuck $\approx$ 174 km/h	Gear schedule regression. Verify <code>Config.aids:</code> <code>rpm_upshift=17800</code> , <code>max_gear=6</code> (and <code>gear_max=7.0</code> — the fixed training normalization constant — in obs config).
TORCS window doesn't auto-start a race (Windows)	<code>autostart_win.py</code> needs the TORCS window focusable; don't lock the screen, check pyautogui/pygetwindow installed.